

LAB EXPERIMENT 11-18

Q.11:

PROGRAM:

```
#include <stdio.h>
#include <string.h>

#define MAX 50

struct Symbol {
    char name[30];
    char type[20];
    char value[30];
};

struct Symbol table[MAX];
int count = 0;

// Insert a symbol
void insert() {
    char name[30];

    if (count >= MAX) {
        printf("\nSymbol table is full!\n");
        return;
    }

    printf("\nEnter symbol name: ");
    scanf("%s", name);

    // Check for duplicate symbol
    for (int i = 0; i < count; i++) {
        if (strcmp(table[i].name, name) == 0) {
            printf("Symbol already exists!\n");
            return;
        }
    }

    printf("Symbol not found!\n");
}

// Display symbol table
void display() {
    if (count == 0) {
        printf("\nSymbol table is empty!\n");
        return;
    }

    printf("\n-----\n");
    printf("          SYMBOL TABLE\n");
    printf("-----\n");
    printf("%-15s %-15s %-15s\n", "Name", "Type", "Value");
    printf("-----\n");

    for (int i = 0; i < count; i++) {
        printf("%-15s %-15s %-15s\n",
            table[i].name,
            table[i].type,
            table[i].value);
    }

    printf("-----\n");
}

// Main function
int main() {
    int choice;
}
```

OUTPUT:

```
===== SYMBOL TABLE OPERATIONS =====
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter your choice: 1

Enter symbol name: count
Enter symbol type: int
Enter symbol value: 10
Symbol inserted successfully!
```

Q.12:

PROGRAM:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

char input[100];
int pos = 0;

void E();
void Eprime();
void T();
void Tprime();
void F();

int error = 0;

void E()
{
    T();
    Eprime();
}

void Eprime()
{
    if (input[pos] == '+')
    {
        pos++;
        T();
        Eprime();
    }
}

void T()
{
    F();
    Tprime();
}

void Tprime()
{
    if (input[pos] == '*')
    {
        pos++;
        F();
        Tprime();
    }
}

void F()
{
    if (input[pos] == '(')
    {
        pos++;
        E();
    }

    if (input[pos] == ')')
    {
        pos++;
    }
    else
    {
        error = 1;
    }
}
```

```

    else if (input[pos] == 'i' && input[pos + 1] == 'd')
    {
        pos += 2;
    }
    else
    {
        error = 1;
    }
}

int main()
{
    printf("Enter the input string: ");
    scanf("%s", input);

    E();

    /* Check whether entire input is consumed */
    if (input[pos] == '\0' && error == 0)
    {
        printf("\nInput string is ACCEPTED.\n");
    }
    else
    {
        printf("\nInput string is REJECTED.\n");
    }

    return 0;
}

```

OUTPUT:

```
Enter the input string: id+id*id
```

```
Input string is ACCEPTED.
```

```
-----
Process exited after 13.91 seconds with return value 0
Press any key to continue . . . |
```

Q. 13:

PROGRAM:

```
#include <stdio.h>
#include <string.h>

char input[100];
int pos = 0;
int error = 0;

/* Function declarations */
void E();
void Eprime();
void T();
void Tprime();
void F();

/* E → T E' */
void E()
{
    T();
    Eprime();
}

/* E' → + T E' | ε */
void Eprime()
{
    if (input[pos] == '+')
    {
        pos++;
        T();
        Eprime();
    }
}

/* Invalid symbol */
else
{
    error = 1;
}

}

int main()
{
    printf("Enter the input string: ");
    scanf("%s", input);

    /* Start parsing from E */
    E();

    /* Check whether the complete string is parsed */
    if (input[pos] == '\0' && error == 0)
    {
        printf("\nThe input string is ACCEPTED.\n");
        printf("The string satisfies the given grammar.\n");
    }
    else
    {
        printf("\nThe input string is REJECTED.\n");
        printf("The string does not satisfy the given grammar.\n");
    }

    return 0;
}
```

OUTPUT:

```
Enter the input string: id+id*id

The input string is ACCEPTED.
The string satisfies the given grammar.

-----
Process exited after 14.61 seconds with return value 0
Press any key to continue . . . |
```

Q.14:

PROGRAM:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main()
{
    char exp[100], temp[10];
    int i, j, len, tempCount = 1;

    printf("Enter an expression: ");
    scanf("%s", exp);

    len = strlen(exp);

    printf("\nThree Address Code:\n");

    /* Process *, / first */
    for (i = 0; i < len; i++)
    {
        if (exp[i] == '*' || exp[i] == '/')
        {
            sprintf(temp, "t%d", tempCount++);

            printf("%s = %c %c %c\n",
                temp, exp[i - 1], exp[i], exp[i + 1]);

            exp[i - 1] = temp[0];
            exp[i] = '\0';

            for (j = i + 2; j < len; j++)
                exp[j - 2] = exp[j];
        }
    }
}
```

OUTPUT:

```
Enter an expression: id+id*id

Three Address Code:
t1 = d * i
t2 = d + i
Result = i

-----
Process exited after 20.23 seconds with return value 0
Press any key to continue . . . |
```

Q.15:

PROGRAM:

```
#include <stdio.h>
#include <ctype.h>

int main()
{
    FILE *fp;
    char filename[100];
    char ch;
    int characters = 0, words = 0, lines = 0;
    int inWord = 0;

    printf("Enter the file name: ");
    scanf("%s", filename);

    fp = fopen(filename, "r");

    if (fp == NULL)
    {
        printf("Unable to open the file.\n");
        return 1;
    }

    while ((ch = fgetc(fp)) != EOF)
    {
        characters++;

        if (ch == '\n')
            lines++;

        if (isspace(ch))
        {
            inWord = 0;
        }
    }
}
```

OUTPUT:

```
Enter the file name: input.txt

Number of Characters: 34
Number of Words: 6
Number of Lines: 2

-----
Process exited after 10.69 seconds with return value 0
Press any key to continue . . . |
```

Q.16:

PROGRAM:

```
#include <stdio.h>
#include <string.h>

struct Instruction
{
    char op[5];
    char arg1[10];
    char arg2[10];
    char result[10];
};

int main()
{
    struct Instruction code[20];
    int n, i;

    printf("Enter number of three-address code instructions: ");
    scanf("%d", &n);

    printf("\nEnter the instructions in the format:\n");
    printf("result = arg1 operator arg2\n");
    printf("For example: t1 = a + b\n\n");

    for (i = 0; i < n; i++)
    {
        printf("Instruction %d: ", i + 1);

        scanf("%s = %s %s %s",
            code[i].result,
            code[i].arg1,
            code[i].op,
            code[i].arg2);

        if (strcmp(code[i].op, "+") == 0)
        {
            printf("MOV R0, %s\n", code[i].arg1);
            printf("ADD R0, %s\n", code[i].arg2);
            printf("MOV %s, R0\n", code[i].result);
        }
        else if (strcmp(code[i].op, "-") == 0)
        {
            printf("MOV R0, %s\n", code[i].arg1);
            printf("SUB R0, %s\n", code[i].arg2);
            printf("MOV %s, R0\n", code[i].result);
        }
        else if (strcmp(code[i].op, "*") == 0)
        {
            printf("MOV R0, %s\n", code[i].arg1);
            printf("MUL R0, %s\n", code[i].arg2);
            printf("MOV %s, R0\n", code[i].result);
        }
        else if (strcmp(code[i].op, "/") == 0)
        {
            printf("MOV R0, %s\n", code[i].arg1);
            printf("DIV R0, %s\n", code[i].arg2);
            printf("MOV %s, R0\n", code[i].result);
        }
        else
        {
            printf("Invalid operator: %s\n", code[i].op);
        }
    }

    return 0;
}
```

OUTPUT:

```
Enter number of three-address code instructions: 1

Enter the instructions in the format:
result = arg1 operator arg2
For example: t1 = a + b

Instruction 1: t1 = a + b

===== THREE ADDRESS CODE =====
t1 = a + b

===== TARGET CODE =====
MOV R0, a
ADD R0, b
MOV t1, R0

-----
Process exited after 24.24 seconds with return value 0
Press any key to continue . . . |
```

Q.17:

PROGRAM:

```
.....
    if (i < count[1] - 1)
        printf(", ");
}
printf(" }\n");

printf("LEADING(F) = { ");
for (i = 0; i < count[2]; i++)
{
    if (leading[2][i] == 'i')
        printf("id");
    else
        printf("%c", leading[2][i]);

    if (i < count[2] - 1)
        printf(", ");
}
printf(" }\n");

it main()

printf("Grammar:\n");
printf("E -> E + T | T\n");
printf("T -> T * F | F\n");
printf("F -> ( E ) | id\n");

computeLeading();
displayLeading();

return 0;
```


OUTPUT:

```
Grammar:
E -> E + T | T
T -> T * F | F
F -> ( E ) | id

===== LEADING SETS =====
LEADING(E) = { (, id }
LEADING(T) = { (, id }
LEADING(F) = { (, id }

-----

Process exited after 5.862 seconds with return value 0
Press any key to continue . . . |
```

Q.18:

PROGRAM:

```
int i;

printf("\n===== TRAILING SETS =====\n");

printf("TRAILING(E) = { ");
for (i = 0; i < count[0]; i++)
{
    if (trailing[0][i] == 'i')
        printf("id");
    else
        printf("%c", trailing[0][i]);

    if (i < count[0] - 1)
        printf(", ");
}
printf(" }\n");

printf("TRAILING(T) = { ");
for (i = 0; i < count[1]; i++)
{
    if (trailing[1][i] == 'i')
        printf("id");
    else
        printf("%c", trailing[1][i]);

    if (i < count[1] - 1)
        printf(", ");
}
printf(" }\n");

printf("TRAILING(F) = { ");
```

```

        printf("%c", trailing[1][i]);

        if (i < count[1] - 1)
            printf(", ");
    }
    printf(" }\n");

    printf("TRAILING(F) = { ");
    for (i = 0; i < count[2]; i++)
    {
        if (trailing[2][i] == 'i')
            printf("id");
        else
            printf("%c", trailing[2][i]);

        if (i < count[2] - 1)
            printf(", ");
    }
    printf(" }\n");

main()

printf("Grammar:\n");
printf("E -> E + T | T\n");
printf("T -> T * F | F\n");
printf("F -> ( E ) | id\n");

computeTrailing();
displayTrailing();

```

OUTPUT:

```

Grammar:
E -> E + T | T
T -> T * F | F
F -> ( E ) | id

===== TRAILING SETS =====
TRAILING(E) = { +, *, ), id }
TRAILING(T) = { *, ), id }
TRAILING(F) = { ), id }

-----
Process exited after 2.828 seconds with return value 0
Press any key to continue . . . |

```